Manual de Seguridad

SH-VT-TEC-001 | Versión 1.0

SECUREHUB / VULNTRACK

PLATAFORMA DE GESTIÓN DE VULNERABILIDADES Y RIESGOS

MANUAL DE SEGURIDAD

Título del proyecto	SecureHub / VulnTrack
Subtitulo	**Plataforma de Gestión de Vulnerabilidades y Riesgos**
Documento	**Manual de Seguridad**
Equipo de Ingeniería	**Muñoz Rey Hilary Michelle** *Analista de Documentación Técnica* **Ruiz Camelo Jennifer Dayan** *Especialista en Documentación de Software* **Molina Lancheros Daniel Steven** *Ingeniero DevSecOps* **Torres Pérez Eduar De Jesús** *Desarrollador de Software* **Alexander Diaz Molano** *Analista Técnico de Arquitectura*
Evaluador Técnico	Ospina Navas Jaider *Magíster en Ciberseguridad y Ciberdefensa*
Fecha de entrega	23 de octubre de 2026
Licenciamiento	MIT License (Código abierto) *Copyright © 2026 Equipo VulnTrack*
Versión del documento	**V1.0 — revisión de evidencias** *(10/10/2026)*

1. Objetivo

Documentar los controles de seguridad que se identifican en el código y la automatización de VulnTrack/SecureHub, la forma de ejecutar o revisar análisis y el proceso de tratamiento de hallazgos. Los reportes reales deben adjuntarse al finalizar cada análisis; este manual no declara resultados que no se hayan aportado.

2. Activos y límites de confianza

- Credenciales, hashes de contraseña, secreto MFA/TOTP y tokens de sesión.
- Datos multiempresa: usuarios, roles, activos, vulnerabilidades, riesgos, planes de remediación y evidencias.
- Base de datos PostgreSQL y políticas de aislamiento por empresa (RLS descrito en README y esquema).
- API REST y frontend; Redis usado para rate limiting y estado/caché.
- Archivos de evidencia, logs, configuración de monitoreo y secretos de integración.
- Fuentes externas: NVD/NIST y SMTP, cuando se configuren.

3. Modelo de amenazas

El repositorio incluye security/threat-model.json. El siguiente resumen es una guía de lectura basada en los componentes y controles descritos por el proyecto; se debe contrastar con el contenido vigente del archivo y los resultados de pruebas.

Amenaza	Impacto potencial	Controles descritos/medidas recomendadas
Acceso no autorizado o escalamiento de privilegios	Exposición o modificación de información de empresas	JWT, middleware de permisos/RBAC dinámico, RLS; probar rutas por rol.
Fuerza bruta contra login	Compromiso de cuentas	Rate limiting apoyado en Redis; verificar límites, respuestas y registros.
Inyección SQL	Lectura/modificación no autorizada	Consultas parametrizadas según README; validar con análisis estático y pruebas.
Fuga de secretos	Compromiso de cuentas, servicios o datos	Gitleaks, .gitignore, variables de entorno; rotar secretos expuestos.
Dependencias o imágenes vulnerables	Ejecución de código o escalamiento	Trivy y actualización controlada de dependencias/imágenes.
Ataques a la aplicación web/API	Exposición de datos o abuso de endpoints	OWASP ZAP en el workflow DAST; probar en staging autorizado.

Pérdida o alteración de datos	Interrupción y pérdida de trazabilidad	Volúmenes persistentes, backups y pruebas de restauración pendientes de formalizar.
Abuso de consolas de monitoreo	Exposición de métricas/logs o cambios de configuración	Restringir puertos y credenciales; no exponer consolas públicamente sin hardening.

4. Controles identificados en el proyecto

- Autenticación basada en JWT y contraseñas procesadas con bcryptjs según la documentación del proyecto.
- MFA/TOTP compatible con aplicaciones autenticadoras.
- Permisos dinámicos y middleware de autorización; aislamiento multiempresa descrito mediante Row-Level Security (RLS).
- Helmet, lista de orígenes CORS, límite de payload JSON y limitación de intentos de inicio de sesión descritos en README.
- Tabla de eventos de auditoría descrita como append-only en la documentación; confirmar restricciones reales en la base de datos antes de considerarla inmutable.
- GitHub Actions: workflows para pruebas backend, seguridad DevSecOps y seguridad de infraestructura.

5. Herramientas de seguridad y configuración observada

Herramienta	Uso identificado	Observación
Gitleaks	Detección de secretos en workflow DevSecOps y pre-commit	Revisar hallazgos y confirmar que los secretos no están activos; la supresión no reemplaza rotación.
Semgrep	SAST con configuración auto sobre backend y frontend	El workflow usa semgrep scan --config auto --error.
Trivy	Escaneo de archivos/dependencias y análisis de imagen en CI; scanner propio con Trivy	El workflow excluye rutas de dependencias/build indicadas en YAML.
OWASP ZAP	DAST de API de staging con reportes HTML y JSON	El workflow genera zap-dast-report.html y zap-dast-report.json.
Jest + Supertest	Pruebas automatizadas de backend	Hay pruebas en backend/tests y workflow de ejecución.

Checkov	Revisión de IaC/infraestructura indicada por workflow de seguridad de infraestructura	Verificar resultados y configuración exacta en el workflow.

6. Ejecución de pruebas y análisis

6.1 Pruebas de backend

```
cd backend
npm ci
npm test -- --ci
```

El workflow tests.yml usa Node.js 22 y ejecuta npm ci y npm test -- --ci. Guardar el log del job de GitHub Actions como evidencia.

6.2 Gitleaks

El workflow incluye la acción gitleaks/gitleaks-action@v3 y el repositorio cuenta con .pre-commit-config.yaml. Para interpretar el resultado, distinguir secretos reales de falsos positivos; cualquier credencial expuesta debe revocarse/rotarse aunque se elimine del historial.

6.3 Semgrep

```
semgrep scan --config auto --error --exclude 'backend/zap-reports/**' backend frontend
```

El comando corresponde al workflow de CI; requiere tener Semgrep instalado o ejecutar el job automatizado. Revisar regla, archivo, línea, severidad y evidencia del flujo.

6.4 Trivy

El workflow DevSecOps incluye un job Trivy y excluye rutas como node_modules, frontend/dist, reportes ZAP y carpetas de agentes. El scanner también se construye desde ./scanner y utiliza Trivy. Los hallazgos deben asociarse al componente, CVE, severidad, versión afectada y versión corregida.

```
docker compose logs --tail=200 securehub-scanner
```

El comando permite revisar el contenedor scanner. La ruta del último JSON se define en Compose como logs/scanner/scanner/latest.json dentro del volumen del scanner; la ubicación en el host depende del montaje y del volumen. No se debe asumir que existe un reporte hasta comprobarlo.

6.5 OWASP ZAP

El workflow ejecuta un escaneo DAST de la API de staging con zap-api-scan.py y publica los reportes HTML/JSON como artefactos. Ejecutar pruebas activas únicamente contra entornos propios o autorizados.

- Confirmar que staging está levantado y que el target corresponde al entorno aprobado.

- Descargar el artefacto vulntrack-zap-dast-report y revisar alertas por riesgo, URL, parámetro, evidencia y solución sugerida.
- Un código de salida no cero no debe interpretarse sin revisar la lógica del workflow y el reporte generado.

6.6 Bandit

En los archivos revisados no se encontró un job o configuración de Bandit. Bandit es una herramienta orientada a código Python; no debe registrarse como ejecutada en este proyecto sin evidencia. Si el equipo debe reportarla por requisito académico, documentar la ausencia como brecha o incorporarla mediante un cambio aprobado, indicando el alcance real.

7. Cómo interpretar los reportes

Campo	Interpretación	Acción
Severidad	Nivel de riesgo asignado por herramienta	Priorizar críticas/altas y exposición pública; no basarse solo en el color.
CVE/CWE/regla	Identificador de vulnerabilidad o categoría de fallo	Comprender causa y aplicabilidad al código/imagen analizados.
Componente y versión	Paquete, imagen, archivo, endpoint o configuración afectada	Verificar que la versión desplegada corresponde al hallazgo.
Evidencia	Fragmento, endpoint, parámetro o cadena detectada	Reproducir en entorno controlado sin exponer datos sensibles.
Remediación sugerida	Actualización, cambio de código o configuración	Validar compatibilidad y probar la corrección.
Estado	Abierto, en análisis, mitigado, corregido, aceptado o falso positivo	Registrar responsable, fecha, justificación y evidencia de reanálisis.

8. Proceso de gestión de vulnerabilidades

1. Registrar cada hallazgo con herramienta, fecha, versión/commit, componente, identificador, severidad y evidencia.
2. Validar si es reproducible y aplicable; documentar los falsos positivos con una justificación verificable.
3. Priorizar según severidad, exposición, criticidad del activo, impacto sobre datos multiempresa y disponibilidad de mitigación.
4. Asignar responsable y fecha objetivo según los SLA aprobados por el equipo. Los tiempos concretos deben ser definidos por el proyecto; no se inventan aquí.

5. Corregir mediante una rama y Pull Request; agregar o actualizar pruebas cuando corresponda.
6. Repetir el análisis que detectó el hallazgo y pruebas de regresión.
7. Cerrar solo con evidencia de corrección o registrar aceptación de riesgo aprobada, con propietario, vencimiento y controles compensatorios.
8. Conservar reporte original, reporte posterior, commit, decisión y fecha en un repositorio de evidencias con acceso controlado.

9. Gestión de incidentes por secretos

1. Revocar o rotar de inmediato el secreto afectado.
2. Determinar alcance, servicios y periodo de exposición.
3. Retirar el secreto del código y del historial cuando sea necesario; la eliminación del archivo no invalida una credencial comprometida.
4. Ejecutar Gitleaks de nuevo y documentar el resultado.
5. Registrar el incidente y medidas preventivas sin copiar el secreto en tickets o reportes.

10. Evidencias requeridas para cerrar el manual

- Exportar resultados reales de Gitleaks, Trivy, Semgrep y ZAP (HTML/JSON o logs del workflow).
- Registrar la fecha, rama, commit, versión de herramientas y entorno analizado.
- Adjuntar resultados de Jest y pruebas de regresión.
- Confirmar si Bandit es un requisito explícito; si lo es, documentar su ausencia o ejecutar una implementación aprobada.
- Completar la matriz de vulnerabilidades con hallazgos reales, responsables, fechas y estado.

11. Limitaciones

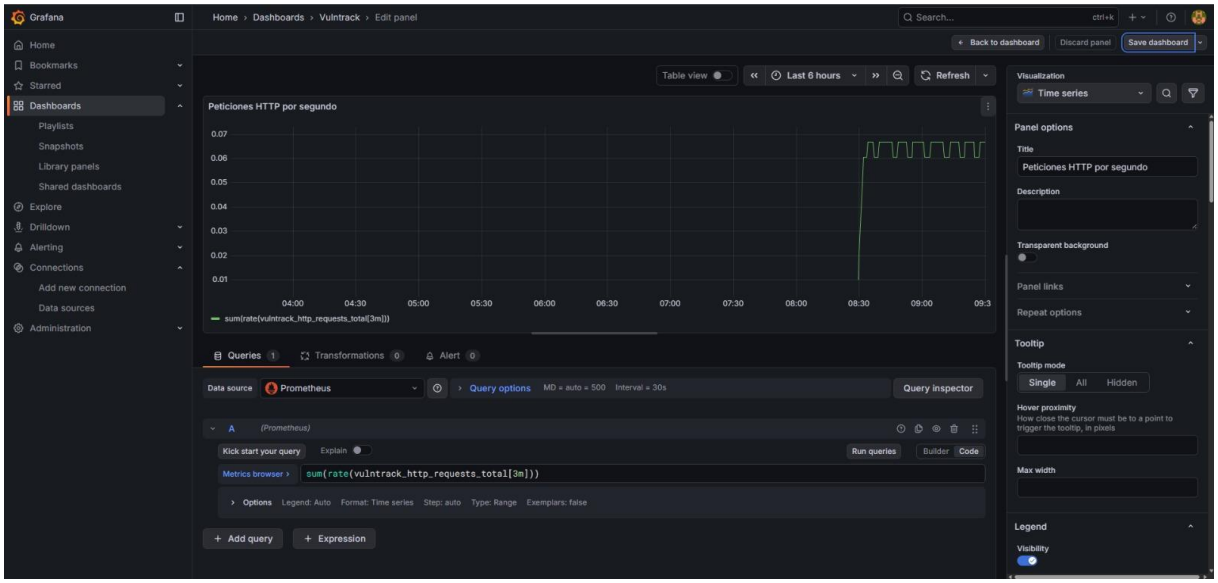
Este documento describe controles y automatizaciones identificados en los archivos compartidos. No constituye certificación de seguridad, no reemplaza una prueba de penetración y no declara que el sistema esté libre de vulnerabilidades. Los resultados deben obtenerse en la versión y entorno de entrega.

9. Monitoreo, observabilidad y evidencias de ejecución

Esta sección documenta las pantallas suministradas de Grafana, Loki, Prometheus y GitHub Actions. Las capturas sirven como evidencia visual del monitoreo y de las ejecuciones mostradas en el momento de la captura. Los estados pueden cambiar entre ejecuciones; por ello, cada validación debe conservar la fecha, el identificador de ejecución y el reporte asociado cuando esté disponible.

9.1 Peticiones HTTP por segundo en Grafana

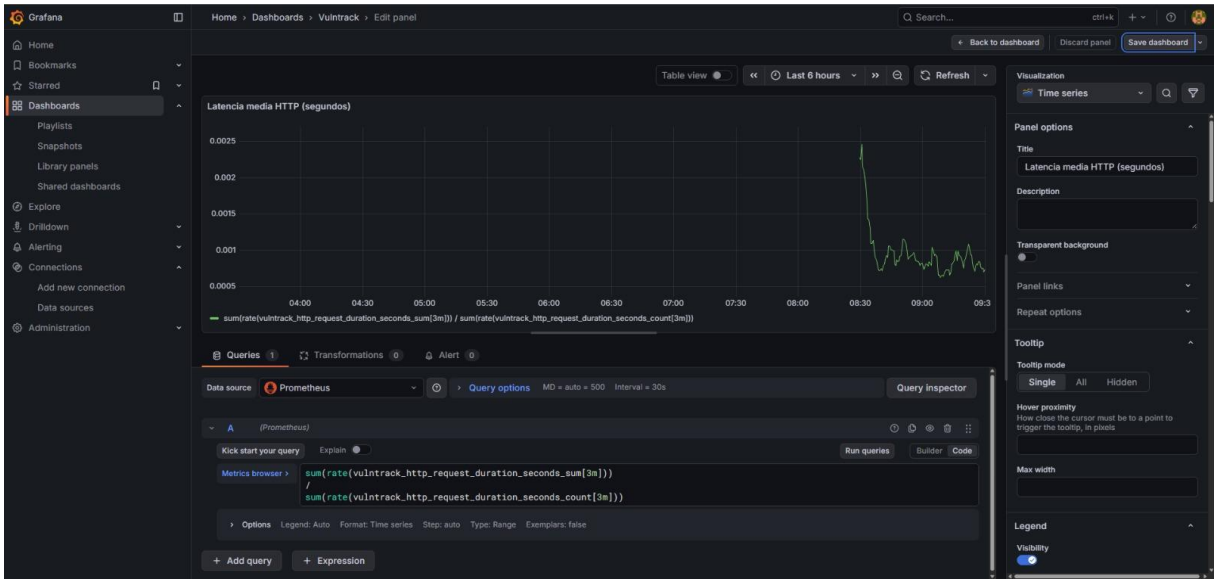
La gráfica presenta la tasa de solicitudes HTTP de la aplicación a partir de la métrica `vulntrack_http_requests_total`, calculada con `sum(rate(vulntrack_http_requests_total[3m]))`. Para revisarla, abra el dashboard VulnTrack, seleccione el panel y confirme que la fuente de datos sea Prometheus. Ajuste el rango temporal y ejecute la consulta si necesita actualizar la visualización. Un aumento sostenido puede justificar revisar logs, consumo de recursos y respuestas HTTP; la gráfica por sí sola no demuestra un incidente.



Evidencia visual: IMG-20261022-WA0009.jpg

9.2 Latencia media de las solicitudes HTTP

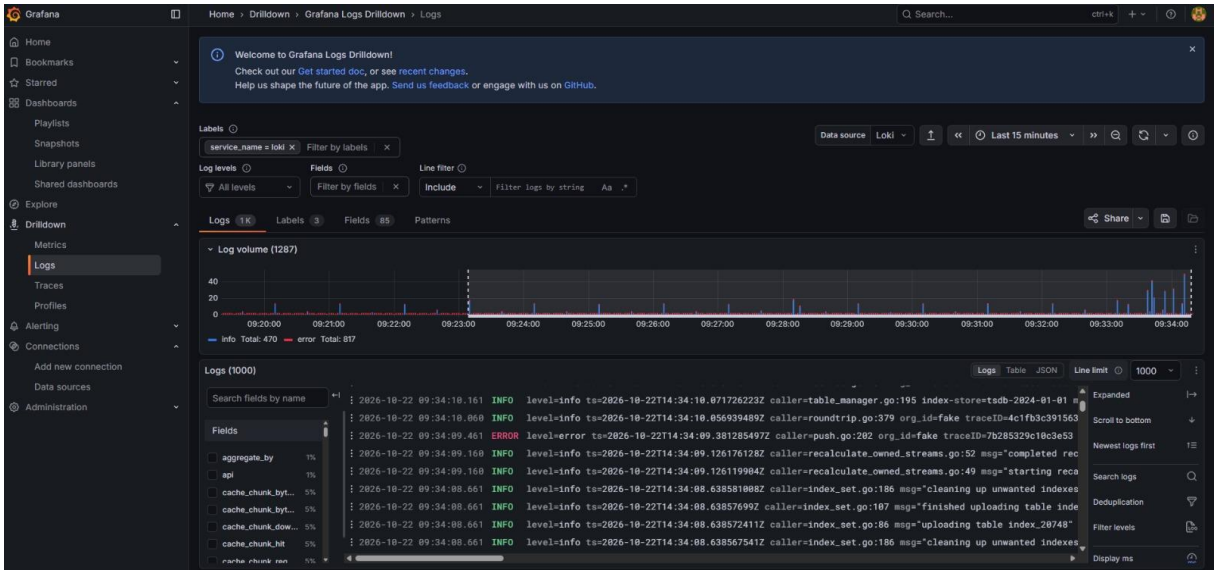
El panel calcula una latencia media aproximada dividiendo la tasa de la suma de duraciones `vulntrack_http_request_duration_seconds_sum` entre la tasa del contador `vulntrack_http_request_duration_seconds_count`, usando una ventana de tres minutos. Revise tendencias y picos dentro del intervalo seleccionado. Compare el comportamiento con la carga y los logs antes de concluir que existe degradación. La captura muestra valores bajos después de un pico inicial, pero no define por sí sola un umbral de aceptación.



Evidencia visual: IMG-20261022-WA0010.jpg

9.3 Consulta de logs en Grafana Loki

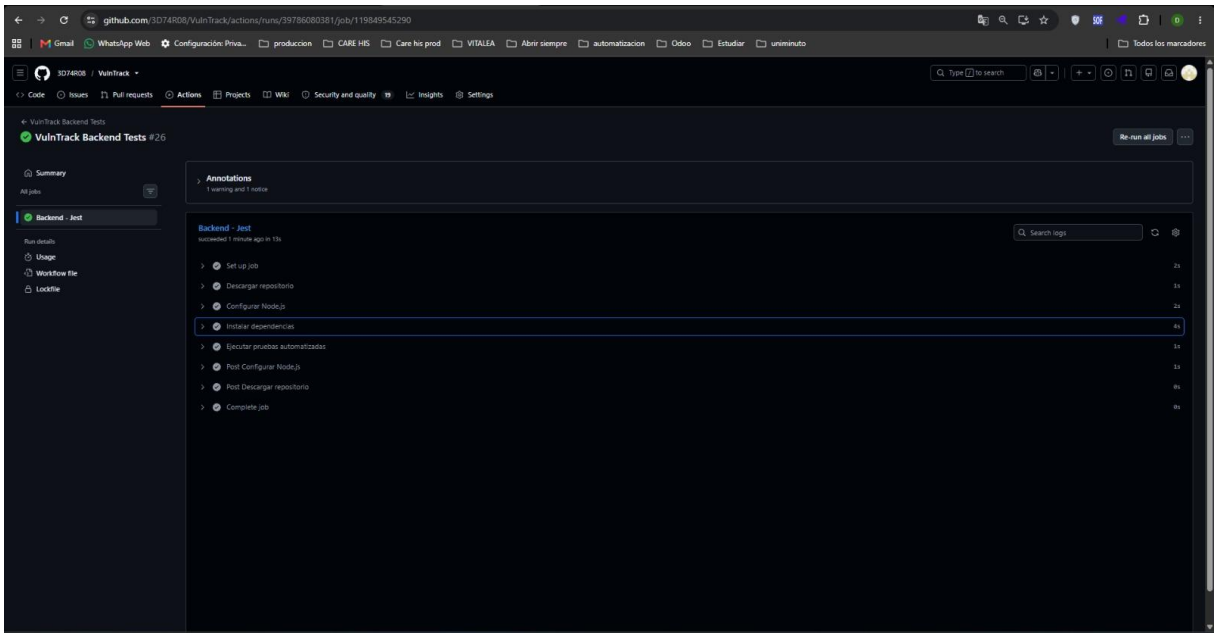
La pantalla permite consultar los registros recopilados por Loki. En la captura se aplica el filtro de etiqueta `service_name = loki` y se muestran niveles, volumen y líneas de log. Para investigar un evento, seleccione el rango temporal, filtre por servicio o nivel, busque palabras clave y abra las entradas relacionadas para revisar la marca de tiempo y el mensaje. La imagen corresponde a logs del servicio Loki; para diagnosticar la API, cambie el filtro al servicio correspondiente. Los registros pueden contener datos operativos sensibles y deben consultarse únicamente con permisos autorizados.



Evidencia visual: IMG-20261022-WA0011.jpg

9.4 GitHub Actions: pruebas automatizadas del backend

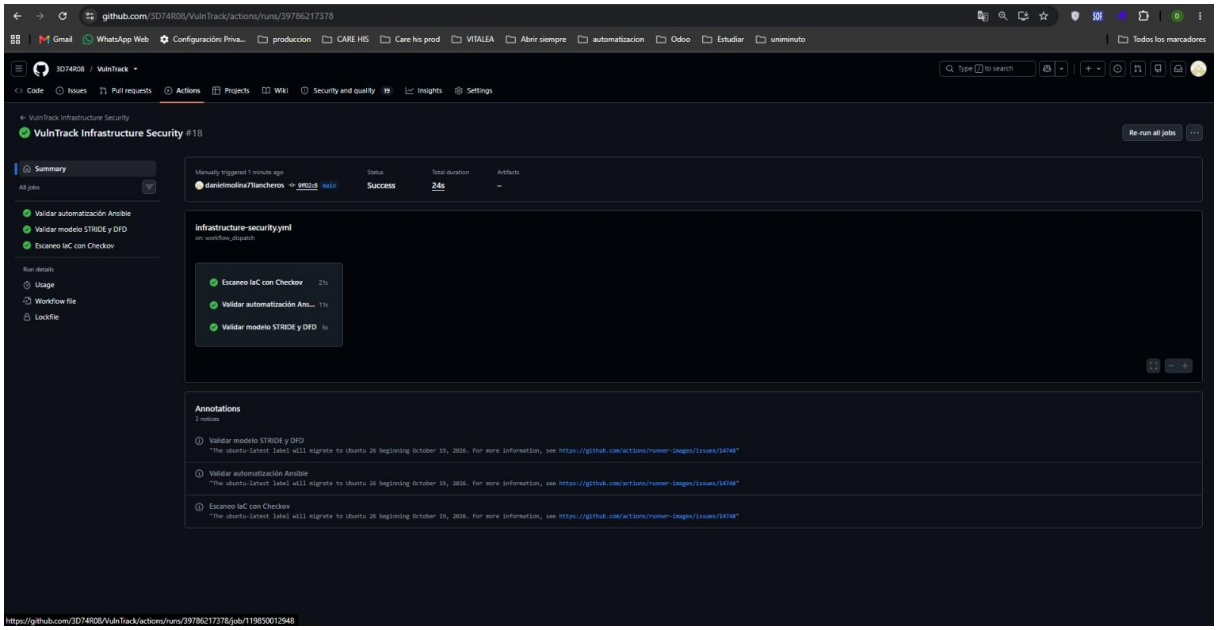
La ejecución mostrada del workflow «VulnTrack Backend Tests» presenta el job «Backend - Jest» en estado satisfactorio. Los pasos visibles incluyen preparación del job, descarga del repositorio, configuración de Node.js, instalación de dependencias y ejecución de pruebas automatizadas. Para validar una ejecución, abra el workflow desde Actions, seleccione el run correspondiente y expanda «Ejecutar pruebas automatizadas» para revisar su salida. Conserve el enlace o identificador del run como evidencia. El estado verde corresponde a esta ejecución específica y no garantiza que todas las versiones futuras estén libres de defectos.



Evidencia visual: IMG-20261022-WA0002.jpg

9.5 GitHub Actions: validaciones de seguridad de infraestructura

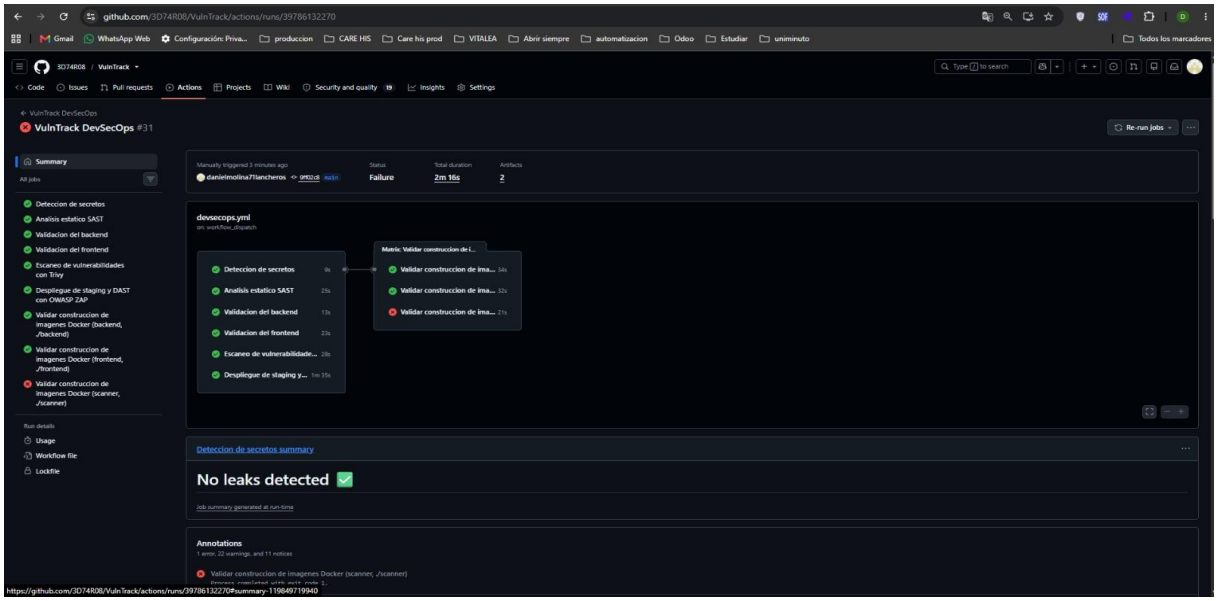
La ejecución del workflow «VulnTrack Infrastructure Security» aparece como satisfactoria. En el resumen se observan validaciones de automatización Ansible, modelo STRIDE/DFD y escaneo de infraestructura como código con Checkov. Para consultar los detalles, abra cada job y revise comandos, advertencias y resultados. La captura también muestra avisos del runner relacionados con la futura migración de la etiqueta `ubuntu-latest`; estos avisos deben evaluarse y atenderse cuando corresponda. Un job exitoso confirma que los pasos configurados finalizaron según sus condiciones, no que se haya demostrado ausencia absoluta de riesgos.



Evidencia visual: IMG-20261022-WA0003.jpg

9.6 GitHub Actions: pipeline DevSecOps y resultado mixto

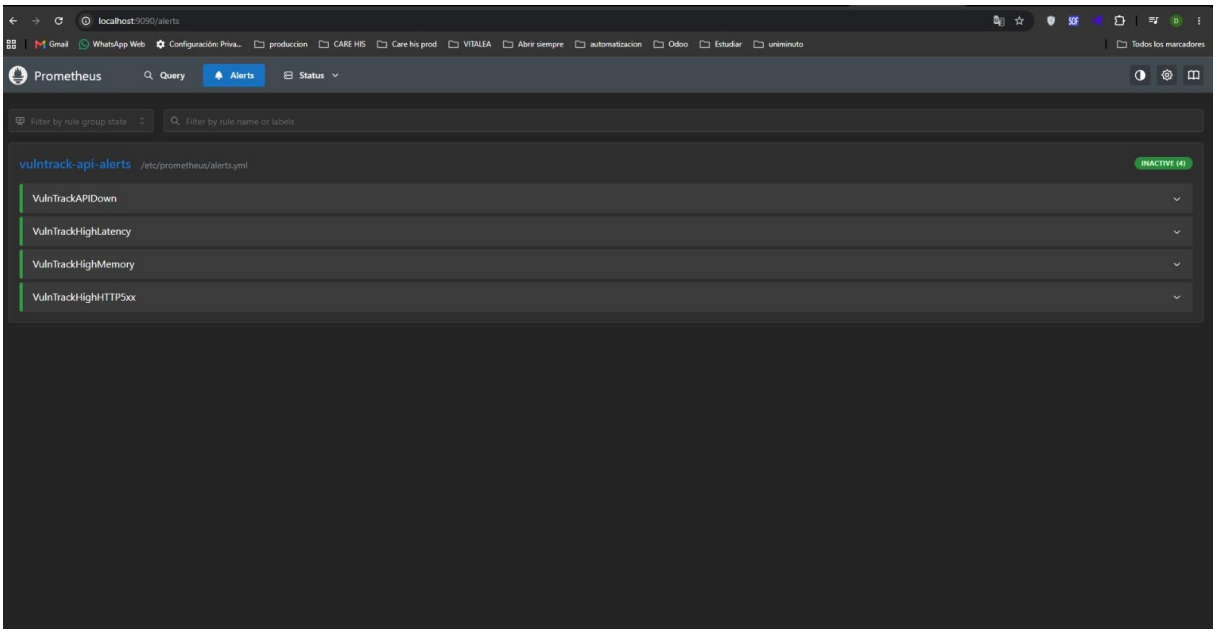
El workflow «VulnTrack DevSecOps» reúne validaciones de detección de secretos, análisis estático, backend, frontend, escaneo con Trivy, despliegue de staging y análisis DAST con OWASP ZAP. En la captura, el paso de construcción de la imagen Docker del scanner aparece en rojo. Según la aclaración del equipo del proyecto, esto ocurrió porque Docker estaba cerrado en ese momento; no corresponde a un defecto persistente del código y la situación ya fue atendida. Para validar el estado actual, ejecute nuevamente el workflow con Docker disponible y revise el resultado del job y sus registros. Conserve la ejecución más reciente como evidencia del estado vigente.



Evidencia visual: IMG-20261022-WA0004.jpg

9.7 Prometheus: estado de las reglas de alerta

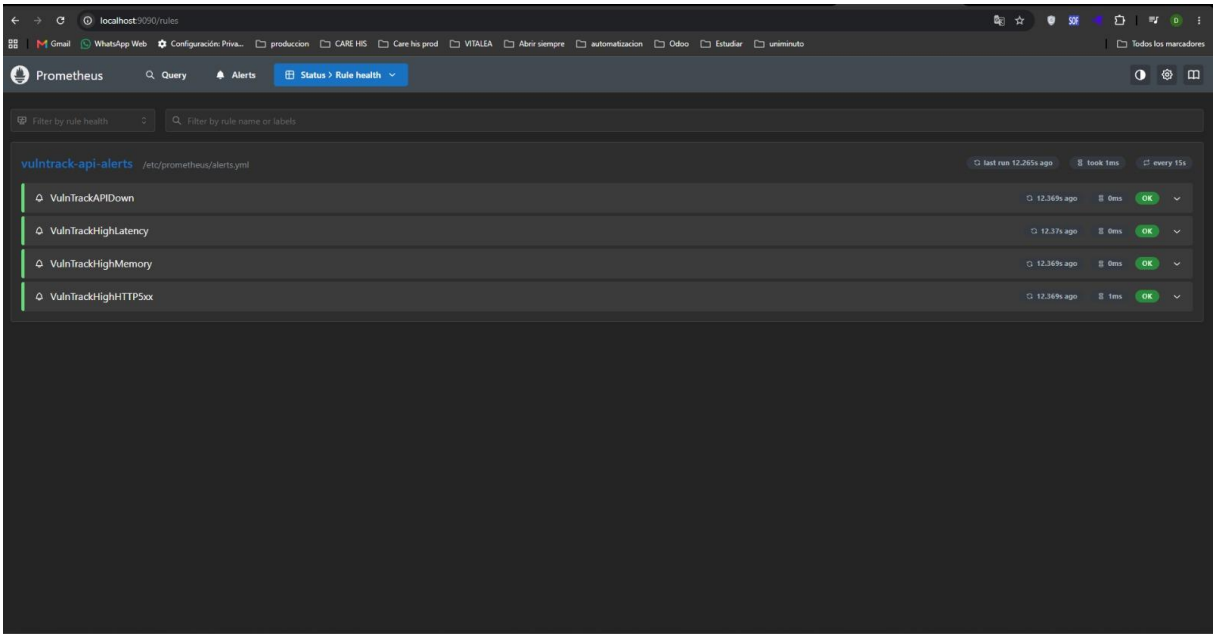
La vista «Status > Rule health» muestra las reglas VulnTrackAPIDown, VulnTrackHighLatency, VulnTrackHighMemory y VulnTrackHighHTTP5xx en estado «OK» en el momento de la captura. Para verificarlas, ingrese a Prometheus, abra el estado de salud de reglas y revise el último cálculo y el intervalo de evaluación. Si una regla cambia de estado o deja de evaluarse, revise la expresión, las métricas de origen y los logs de Prometheus. El estado «OK» indica que la regla se evalúa correctamente; no equivale a una alerta activa.



Evidencia visual: IMG-20261022-WA0005.jpg

9.8 Prometheus: vista de alertas configuradas

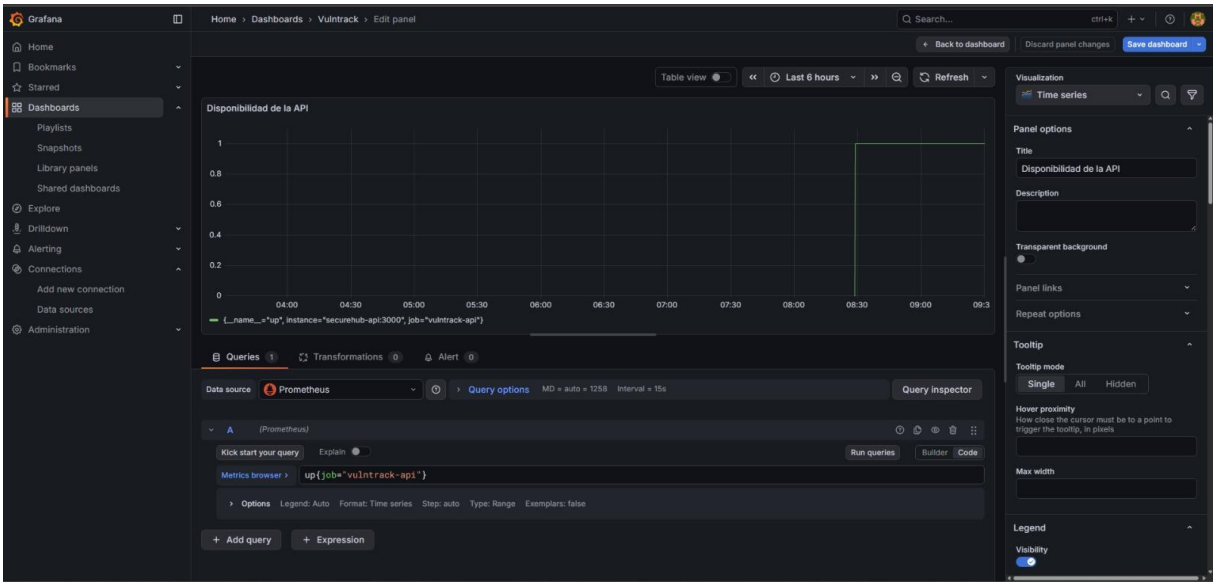
La vista «Alerts» presenta las cuatro reglas de alerta configuradas y el grupo `vulntrack-api-alerts`. En la captura, el grupo indica «INACTIVE (4)», es decir, no hay alertas activas para esas reglas en ese instante. Para comprobar una alerta, despliegue su fila y revise la expresión, las condiciones y el estado. Si se requiere probar el envío de notificaciones, documente por separado la configuración del gestor de alertas y el resultado de una prueba controlada; esta pantalla no demuestra que exista un canal de notificación operativo.



Evidencia visual: IMG-20261022-WA0006.jpg

9.9 Grafana: disponibilidad de la API

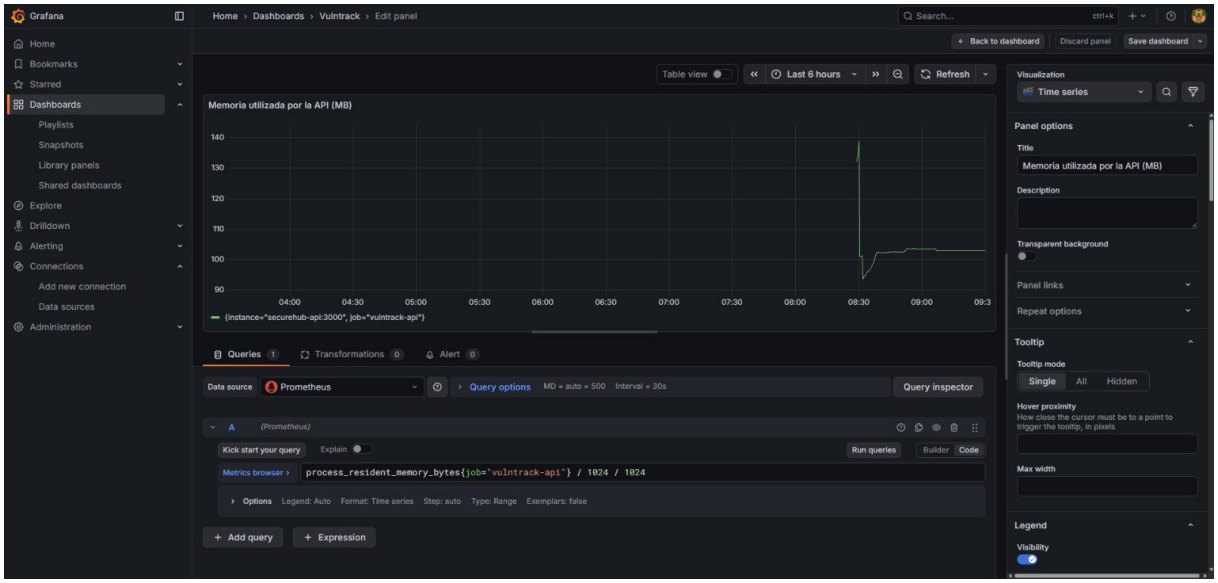
El panel consulta `up{job="vulntrack-api"}` desde Prometheus. El valor 1 indica que Prometheus puede alcanzar y recopilar métricas del objetivo identificado en esa serie; el valor 0 indica que no logra recopilarlo. En la captura, la serie pasa a 1 aproximadamente a las 08:30 y permanece en ese valor durante el tramo visible. Para investigar una caída, revise la configuración del scrape, la conectividad, el estado del contenedor y los logs de la API. Esta métrica verifica el objetivo de monitorización, no todos los flujos funcionales de la aplicación.



Evidencia visual: IMG-20261022-WA0007.jpg

9.10 Grafana: memoria utilizada por la API

El panel muestra la métrica `process_resident_memory_bytes{job="vulntrack-api"} / 1024 / 1024`, convertida a megabytes. Utilice el rango temporal para observar tendencias y picos, y compare el consumo con los límites de memoria del contenedor y la carga de trabajo. Un pico aislado no demuestra una fuga de memoria; si el consumo crece de forma sostenida, correlacione con reinicios, logs, tráfico y métricas adicionales.



Evidencia visual: IMG-20261022-WA0008.jpg